

Proyecto 1 De Estructuras de datos, Documentación – Sem Mora Chinchilla

Relación entre las estructuras.

En mi proyecto todas las estructuras se relacionan por medio de la clase Juego. Las estructuras o clases no se comunican entre ellas directamente, sino que la clase Juego es la que tiene a todas y decide cuándo usar cada una. La parte gráfica lo realizan los archivos de Dibujo .cpp/.h y Pantallas .cpp/.h solo le piden los datos a la clase de Juego para dibujarlos con la librería gráfica de raylib.

Al empezar una partida, mi clase de Juego llena la cola de piezas (ColaPiezas) con una bolsa de las 7 piezas mezcladas al azar, y cada vez que en la cola quedan menos de 3 piezas se agrega otra bolsa, así siempre se pueden mostrar las 3 siguientes y nunca se acaban las piezas. Cuando una pieza cae, la clase de Juego escribe sus bloques en el tablero, el cual es una lista enlazada de 20 nodos donde cada nodo es una fila con un arreglo de 10 celdas. Después revisa si hay filas completas y si las hay, primero parpadean y luego se eliminan esos nodos y se insertan filas vacías al inicio de la lista. Luego se desencola la siguiente pieza y aparece arriba del tablero. Si el jugador presiona la tecla C, la pieza actual que va bajando, se mete en la pila del hold (PilaHold), que tiene capacidad de 1 pieza a la vez, y si ya había una pieza guardada se saca y pasa a ser la pieza actual. El hold lo establecí para poderse usar solo una vez por pieza.

Para el historial uso una lista doblemente enlazada llamada Historial. Cada vez que pasa algo importante como, por ejemplo: mover, rotar, bajar, colocar una pieza o usar el hold. La clase de Juego guarda un nodo con una copia del estado de la partida en ese momento lo que guarda es: el tablero en una matriz de 20x10, la pieza actual, el hold, las piezas siguientes, el puntaje, las líneas y el nivel. Con las teclas de Z y X se deshace y se rehace los movimientos, funciona moviendo el puntero de la lista y la clase Juego carga ese estado. Si después de deshacer se hace una jugada nueva, los pasos que estaban adelante se borran ya que no son parte de la cronología original por decirlo de una manera. Cuando se pierde, esa misma lista se recorre desde el primer nodo hasta el final, para poder ver la repetición paso a paso.

La cola de eventos llamada ColaEventos está ordenada por el momento en el que se tiene que disparar cada evento. En cada frame, la clase Juego revisa si al evento del frente ya le toca según el tiempo de la partida, y si es así lo saca y aplica su efecto. Por último, la tabla de mejores puntajes es una lista simple llamada ListaPuntajes que se carga del archivo puntajes.txt al abrir el

juego. Al terminar una partida, si el puntaje entra al top 10 se pide el nombre del jugador, se agrega a la lista, se ordena con el algoritmo que el usuario escogió, los cuales pueden ser Bubble Sort o Merge sort, se quitan los que ya no pertenecen al top de 10 y se vuelve a guardar en el archivo.

Eventos y puntaje.

Los eventos que programé son tres. El de velocidad pasa a los 30 segundos iniciales y después cada 30 segundos, sube el nivel y hace que las piezas caigan más rápido. El de puntos dobles empieza a los 45 segundos y dura 10 segundos; el inicio y el fin son dos eventos separados dentro de la cola para saber cuándo empieza o finaliza ya que es el único evento que dura cierto tiempo, y cuando se acaba se programa el siguiente para 45 segundos después. El de la bomba pasa a los 20 segundos y luego cada 40 segundos, y hace que la siguiente pieza sea una bomba (sale blanca con un punto rojo para poderla diferenciar de las piezas normales) que al caer no se queda en el tablero, sino que borra la fila que tiene justo abajo.

Para el puntaje usé la fórmula clásica: 1 línea da 100 puntos, 2 líneas 300, 3 líneas 500 y 4 líneas 800. Mientras están activos los puntos dobles el resultado se multiplica por 2, y la bomba cuenta como si fuera 1 línea ósea +100 puntos por bomba.

Complejidad teórica.

En la siguiente tabla (n) es la cantidad de nodos de cada estructura. En el caso del tablero n siempre es 20 (las filas), así que en la práctica sus operaciones son rápidas, pero lo dejé en función de n para que se vea de qué depende cada una de las complejidades.

Clase	Operación	Mejor caso	Peor caso
ColaPiezas	encolar (tengo un puntero al último)	$O(1)$	$O(1)$
ColaPiezas	desencolar	$O(1)$	$O(1)$
ColaPiezas	verPosicion (ver las siguientes)	$O(1)$	$O(n)$
Bolsa	agregarBolsa (mezcla las 7 piezas y las encola)	$O(1)$	$O(1)$
PilaHold	apilar y desapilar	$O(1)$	$O(1)$
Tablero	insertar una fila vacía al inicio	$O(1)$	$O(1)$
Tablero	buscar una fila (para leer o poner una	$O(1)$	$O(n)$

Clase	Operación	Mejor caso	Peor caso
	celda)		
Tablero	limpiarLineas (recorrer, borrar las completas e insertar vacías)	$O(n)$	$O(n)$
Tablero	eliminarFila (la que usa la bomba)	$O(1)$	$O(n)$
Historial	agregar un paso	$O(1)$	$O(n)$
Historial	retroceder y avanzar	$O(1)$	$O(1)$
ColaEventos	insertar ordenado	$O(1)$	$O(n)$
ColaEventos	sacar y ver el frente	$O(1)$	$O(1)$
ListaPuntajes	agregar	$O(1)$	$O(1)$
ListaPuntajes	entraAlTop	$O(1)$	$O(n)$
ListaPuntajes	recortar, cargar y guardar	$O(n)$	$O(n)$
Ordenamiento	BubbleSort	$O(n)$	$O(n^2)$
Ordenamiento	mergeSort	$O(n \log n)$	$O(n \log n)$

En agregar del historial el peor caso es cuando se hace una jugada nueva después de deshacer, porque primero hay que borrar los pasos que estaban adelante. En BubbleSort el mejor caso es cuando la lista ya está ordenada, porque en la primera pasada no hay ningún cambio y termina.

Tabla de tiempos de ordenamientos.

Estos tiempos los saqué con la opción de medir tiempos del menú (tecla M). El programa llena la lista con puntajes al azar, cada tiempo es el promedio de 3 ciclos enteros y los dos algoritmos ordenan exactamente los mismos datos porque uso la misma semilla aleatoria. Los tiempos están en microsegundos y también quedan guardados en tiempos.txt por si se quieren revisar, eso sí cada que se accede al menú de tiempos ósea tecla M, estos tiempos pueden variar por diferentes circunstancias una por ejemplo puede ser el poder y velocidad de procesamiento de la pc que esté ejecutando el programa, pero de igual forma esta tabla permite ver un aproximado de la diferencia entre métodos con diferentes cantidades de datos.

Cantidad de datos	Bubble Sort $O(n^2)$	Merge sort $O(n \log n)$
10	0.97	0.43
100	78.63	4.43

Cantidad de datos	Bubble Sort $O(n^2)$	Merge sort $O(n \log n)$
1000	8129.93	61.20
10000	824210.13	835.67

Preguntas de análisis obligatorias.

1. Compare el tiempo de ordenar la tabla de puntajes con el algoritmo de complejidad $O(n^2)$ contra el de $O(n \log n)$, para tamaños crecientes de datos de prueba. ¿A partir de qué tamaño se nota la diferencia? ¿Coincide con lo que predice la notación asintótica?

R/= Con 10 datos los dos tardan menos de un microsegundo, así que la diferencia no se nota. Con 100 datos BubbleSort ya tarda unas 18 veces más que MergeSort, pero los dos siguen siendo rapidísimos. Donde la diferencia ya se nota de verdad es a partir de los 1000 datos, porque Bubblesort tarda unos 8 milisegundos y MergeSort apenas 0.06. Con 10000 datos BubbleSort duró casi un segundo (unos 824 mil microsegundos) mientras que MergeSort ni llegó a un milisegundo, o sea que fue casi 1000 veces más rápido. Esto sí coincide con la notación asintótica: cada vez que los datos se multiplican por 10, el tiempo de BubbleSort se multiplica más o menos por 100, que es lo que se espera de un $O(n^2)$, y el de MergeSort crece unas 13 o 14 veces, que es más o menos lo que da $n \log n$, en resumen desde los 100 datos ya es visible la diferencia entre algoritmos y sí coinciden con lo que predice la notación asintótica.

2. Explique por qué representar el tablero como una lista enlazada de filas es una forma razonable de modelar la limpieza de líneas, y qué costo (en operaciones) tiene en su implementación insertar una fila vacía y eliminar una fila completa.

R/= Es razonable porque limpiar una línea es básicamente sacar un nodo de la lista, y como las demás filas están enlazadas, las de arriba “bajan” solas cuando se inserta una fila vacía al inicio. Con una matriz habría que copiar todas las filas de arriba una posición hacia abajo cada vez que se limpia una línea lo cual es mucho más ineficiente. En mi implementación insertar una fila vacía cuesta $O(1)$, porque solo creo el nodo con sus 10 celdas en 0 y lo pongo de primero en la lista. Para eliminar una fila completa voy recorriendo la lista con un puntero al nodo anterior, y cuando encuentro la fila completa solo hago que el anterior apunte al siguiente para saltarla y borro el nodo, así que quitarla es $O(1)$. Lo que sí cuesta es el recorrido para encontrarla, que es

$O(n)$, pero como n son 20 filas es muy poco costo pero si n fuera más de 10000 los recursos y la complejidad sí serían mayores y posiblemente con miles de datos pueda provocar problemas .

3. ¿Por qué el replay requiere una lista doblemente enlazada y no una simplemente enlazada? ¿Qué costo tiene, en su implementación, retroceder o avanzar un paso?

R/= Porque en la repetición y en el deshacer y rehacer hay que moverse para los dos lados tanto adelante como atrás. Con una lista simple solo se puede ir hacia adelante, y para retroceder un paso habría que recorrer otra vez la lista desde el inicio hasta el nodo anterior, lo que sería $O(n)$ cada vez. Con la lista doble cada nodo tiene un puntero al anterior y otro al siguiente, entonces retroceder o avanzar un paso cuesta $O(1)$, porque solo muevo el puntero actual al anterior o al siguiente y cargo ese estado siendo mucho más versátil y la opción correcta para el replay.

4. ¿Cómo garantiza su implementación que la cola de eventos programados siempre mantenga al frente el evento más próximo a dispararse? ¿Qué complejidad tiene insertar un nuevo evento?

R/= Lo garantizo desde que el evento se inserta. En vez de meterlo al final, recorro la cola desde el frente hasta encontrar el primer evento que tenga un momento mayor y lo meto justo antes , ósea como un insertar de forma ordenada, entonces la cola siempre queda ordenada de menor a mayor y el evento más próximo siempre está de primero en la cola. Por eso para saber si ya toca un evento solo reviso el del frente, y desencolarlo me cuesta apenas $O(1)$. Al empezar la partida inserto los eventos en desorden (30, 45 y 20 segundos) a propósito, y el primero que sale es el de 20, lo que confirma que la cola solita los va acomodando. Insertar un evento es $O(n)$ en el peor caso, cuando le toca irse al puro final, y $O(1)$ en el mejor caso, cuando queda de primero. En la práctica la cola nunca tiene más de 3 o 4 eventos por dentro.